

LabelHub AI Coding Process

LabelHub AI Coding Process

提交人: Zhang Youchen

项目: LabelHub 数据标注平台

日期: 2026-06-05 (2026-06-10 增补第 9 章「交付前硬化与第二轮 Review」)

1. 目标与验收标准

LabelHub 的目标不是做一个静态管理后台,而是交付一条可被评委独立跑通的数据标注生产链路:

- Owner 能完成建任务、搭模板、发布、查看统计、导出结果。
- Labeler 能完成领任务、作答、提交、看到打回、修改重提。
- AI Agent 能在提交后自动预审,并把 verdict、score、维度评分、prompt version、token 与延迟信息留痕。
- Reviewer 能基于 AI 预审结果做人工初审、复核、通过或打回。
- 导出结果能覆盖 JSON、JSONL、CSV、XLSX、Markdown,供下游继续消费。

AI Coding 的使用原则是:让模型帮助快速盘点实现面、生成测试与发现遗漏,但关键状态机、权限、幂等、导出安全等决策由工程约束反推,不让模型凭空扩展需求。

2. 从原型到可运行主链路

早期实现先收敛最小主链路:用户角色、任务、题目、模板、提交、审核、导出。AI 辅助在这一阶段主要用于拆解接口边界与生成初版测试用例。

关键取舍:

- 用 Go + Gin + GORM 承担 API 与事务边界,避免把核心状态机散落在前端。
- 前端保留 React + TypeScript SPA,按 Owner / Labeler / Reviewer 角色组织工作台。
- 模板系统先做自有 Schema Renderer 与 Designer,覆盖 ShowItem、Radio、Tags、Input、TextArea、RichText、JSONEditor、FileUpload、LLMTrigger,再扩展 Group/Tabs。
- 所有提交与审核动作都落库,不依赖浏览器临时状态。

AI 参与的主要价值是把散乱需求转成可执行检查表:哪些状态需要锁、哪些角色可见、哪些路径需要测试,以及哪些演示步骤会被评委直接碰到。

3. 状态机与事务纪律

LabelHub 把任务与提交状态集中在 `apps/api/internal/statemachine`, 所有业务服务只通过合法 event 推进状态。

提交主路径:

- `draft` -> `submitted`
- `submitted` -> `ai_reviewing` 或 `human_reviewing`
- `ai_reviewing` -> `human_reviewing` / `manual_review` / `revising`
- `manual_review` -> `human_reviewing` / `rejected` / `revising`
- `human_reviewing` -> `approved` / `rejected` / `revising`
- `revising` -> `submitted`

AI Coding 在这部分主要用于补齐非法转移测试矩阵。后续人工 review 又发现旧文档仍保留 `ai_auto_approved`; 最终实现改为 AI 不自动入库, 无论 AI pass 还是 uncertain 都需要人工路径完成最终决策。

工程约束:

- 审核动作事务内锁 submission 与 task item。
- 并发审核使用 `RowsAffected` 检查防双写。
- 打回后 labeler 修改会创建新 revision, 旧 AI 与人工结果保留可追溯。

4. AI Agent 预审设计

AI 预审不是同步 HTTP 调用,而是 durable outbox + Asynq worker。

API 提交流程:

- 保存 submission revision。
- 写入 `ai_reviews` pending 记录。
- 同事务写 `outbox_events(ai:review)`。
- outbox publisher 把事件投递到 Redis / Asynq。

Worker 消费流程:

- 读取当前 submission、revision、task baseline、active AI prompt。
- 调用 mock 或 OpenAI-compatible / Doubao provider。
- 校验 verdict、overall_score、dimensions schema。
- 写回 `ai_reviews` 结果和 audit log。
- 按 verdict 推进 submission 状态。

默认本地环境使用 deterministic mock,便于评委无 API key 跑通;真实环境只需切换

`LLM_PROVIDER`、`LLM_BASE_URL`、`LLM_API_KEY`、`LLM_MODEL`。

5. Prompt、Golden Sample 与 Dry-run

Owner 侧 AI Prompt 不是硬编码规则,而是任务级配置:

- Prompt template
- 维度列表
- pass threshold
- uncertain min
- model
- active prompt id

Golden Sample 用来在正式标注前试跑 prompt。每条样例包含 payload、expected answer、expected verdict,worker 写回 actual verdict 与 matched result。

AI Coding 在这部分帮助快速生成了接口边界与前端轮询状态,但最终实现保留了几个明确限制:

- dry-run 有 task scoped quota / circuit guard。
- prompt version 固化在 AI review 与 dry-run 记录里。
- Reviewer 只能查看规则,不能直接修改 Owner 的 prompt。
- Seed 官方任务现在自带 active prompt 与 golden samples,保证 5 分钟评委路径无需额外手工配置。

6. 多格式导出与下游消费

导出由 `pkg/exporter` 作为共享模块实现,API 与 worker 复用同一套编码逻辑。

支持格式:

- JSON
- JSONL
- CSV
- XLSX
- Markdown

关键安全与稳定性处理:

- 导出是异步 job,状态从 `queued` -> `running` -> `succeeded` / `failed`。
- worker 使用临时文件写入,完成后原子 `rename`。
- 下载走 HMAC 签名 URL 与过期时间。
- CSV 对 `= + - @` 等开头单元格加前导单引号,防公式注入。
- API 启动时要求 `EXPORT_DIR` 是绝对路径,避免 api/worker CWD 不同导致下载失败。

AI 辅助主要用于 review exporter 边界:错误是否可重试、CSV 注入、签名下载、path traversal、前端 stale polling。

7. 测试与工程质量

测试层级覆盖三类风险:

- 纯逻辑单测:状态机、review service、outbox publisher、exporter encoder、AI verdict mapping。
- Handler 单测:权限、参数校验、错误码、owner/labeler/reviewer 主接口。
- Testcontainers 集成测试:真实 MySQL + Redis 覆盖 submit -> outbox -> AI -> revise -> resubmit -> approve -> export。

前端开启 TypeScript strict mode,并用 Vitest 覆盖:

- Schema Renderer 的 ShowItem、Tabs、LLMTrigger。
- Designer 保存 schema 后再由 Renderer 解析的 round-trip。
- Labeler 自动保存、快捷提交、打回修改。
- Owner AI Prompt、Golden Sample、导出、Stats Board。
- Reviewer 队列与 AI verdict 展示。

CI 跑 Go test / vet、web test / lint / build,并单独跑 integration job。

8. 产品体验与最后收敛

最后阶段重点不是继续加功能,而是清理评委会直接遇到的阻塞:

- README 账号密码必须和 seed 一致。
- 官方任务 seed 后必须有 active AI Prompt,否则 labeler 提交不会触发 AI review。
- 架构图必须反映真实 worker 行为,不能保留已删除的 auto approve。
- 交付包 manifest 必须指向真实存在的文件,不能把待补素材伪装成交付物。
- 1280 和 1920 视口下 Designer、Owner、Reviewer 关键路径不能出现遮挡或布局坍塌。

AI Coding 的最终价值体现在这些收敛环节:用 review checklist 反复对照实现、文档、demo 脚本、seed 数据,把“代码能跑”和“评委能独立验收”之间的差距补齐。

9. 交付前硬化与第二轮 Review

主链路 with 交付包成型后,又做了一轮交付前硬化。这一阶段最能体现”AI 辅助 + 工程约束反推”的协作方式:让多路 reviewer agent 并行扫盘,但每条结论都用代码证据复核后才落地,既不过真问题,也不被误报带偏。

两轮 Review,逐条核验

第一轮 4 路并行 review(Go / TypeScript / 安全 / 交付物 readiness)产出 Critical→Minor 共约 40 条,据此修掉 8 个 Critical 与 10 个 High:导出目录绝对路径校验、并发审核

`RowsAffected` 守卫、审核队列 `Limit`、决策按钮 loading guard、Golden Sample 轮询 cleanup、生产日志降噪(`zap.NewProduction`)、上传 `MaxBytesReader`、原生 `confirm` 换 Semi `Modal`、嵌套字段 `_draftId` 防拖拽焦点丢失等。

针对 S7/S8 新增代码(labeler 大任务领取、AI 审核队列视图、acceptance 复核)又做了第二轮针对性 review,发现 6 处并修复,其中 1 处是 Critical 级 IDOR: `GET /reviewer/ai-reviews` 缺任务范围限制,任一 reviewer 可枚举所有任务的 AI review 与 prompt 配置——修复方式是 join `submissions` 并套用 `task_reviewers` scope,且补 handler 测试断言 scope join 存在以防回归。其余包括:打回后队列计数漏 `superseded_at IS NULL`、游标分页在非唯一排序下跳/重(改复合游标)、worker 对不可解析 payload 静默成功(改 `SkipRetry` 让死队列可见)、导出失败被吞导致卡在 `running`、AI verdict 面板硬编码阈值改读 prompt 配置。

误报:改前先验证

第二轮 review 中,reviewer agent 还报了 3 个”Critical”,但逐一对照代码后判定为误报,没有改动:sweeper 的”半提交”由 `FOR UPDATE SKIP LOCKED` 保护、SchemaRenderer 的”无限循环”因 `pruneHiddenAnswerValues` 在值未变时返回同一引用而不成立、TabsField 的”每次击键跳变”因 effect 依赖是按值比较的原始值而不会发生。这一步是 AI Coding 流程里很重要的纪律:模型的高置信报告也必须用代码证据复核,误报照改会引入新风险。

性能与稳定性

- 5000 题线上压测:API 侧无瓶颈(import 5.4s、`GET labeler/items` 全量 ≈468KB 仅 0.7s),瓶颈在前端一次性渲染近 2 万个 DOM 节点。据此给题目导航做了定高虚拟滚动(`visibleRange` 纯函数 + 定窗渲染),大任务下保持流畅。压测数据测后即清。
- AI 审计可追溯:AI 预审写回的审计记录现以 seed 的 `system_ai` 账号身份落库(`actor_type='ai_worker'` + `actor_id`),让”谁做的这条审核”在人工与 AI 之间一致可查。
- 测试链路根因修复:本机 vitest 长期”卡死”,此前误归因为内存。最终定位是 `vitest.config.ts` 的 `pool: 'vmForks'` 在 node 24 上做 vm 隔离编译病态地慢(单文

件近 929s);改为 `pool: 'threads'` 后同文件 6.4s,此前从未跑成的套件全绿。教训同样是:不靠猜测,用最小复现(`new happy-dom Window` 仅 137ms)把根因钉死在 vmForks 层。

最终交付状态:Critical/High 全修并部署,全量 Go 测试绿,服务端 `tsc -b && vite build` 干净,prod 与本地 HEAD 一致。唯一硬缺口是演示视频实录。